

AI assistant : intent classification explained

1. [Q: Explain intent classification and how LLMs can be used](#)
2. [Q: How LLMs can be used for intent classification](#)
3. [Q: What is the relation to multiturn conversations & tool/function calling?](#)
 - a. [Relation to multi-turn conversations](#)
 - b. [Relation to tool / function calling](#)
 - c. [How LLMs enable both in practice \(concise\)](#)
4. [Q: If a business, product or management stakeholder expects intent classification to be used at the first prompt/query from the user, how to explain and manage the expectations?](#)
 - a. [Key message \(one line\)](#)
 - b. [How to explain the limits \(bullet points\)](#)
 - c. [Recommended expectation management \(concrete steps\)](#)
 - d. [Short example phrasing for stakeholders](#)
5. [Q: List more possible intent outcomes](#)
6. [Q: Is intent classification always followed by actioning the intent?](#)
7. [Q: Compare intent classification with the classification task in machine learning](#)
8. [Q: Elaborate with concrete functional examples / use cases making the case for "context dependence" and "actions, clarifications, tool calls, human handoff; must validate slots and permissions"](#)
 - a. [Example 1 — Ambiguous first turn that needs context](#)
 - b. [Example 2 — Multi-turn disambiguation that changes intent](#)
 - c. [Example 3 — Slot filling and validation before action \(financial\)](#)
 - d. [Example 4 — Tool chaining with intermediate clarifications](#)
 - e. [Example 5 — High-risk automatic actions vs. human handoff](#)
 - f. [Example 6 — Context changes intent \(intent drift\)](#)
 - g. [Practical implementation patterns](#)

9. [Q: Provide best ways to implement context and orchestration/flow.](#)

- a. [1\) Represent context reliably](#)
- b. [2\) Encode context for models](#)
- c. [3\) Use hybrid modeling for intent + slots](#)
- d. [4\) Confidence, thresholds, and policies](#)
- e. [5\) Orchestration / controller responsibilities](#)
- f. [6\) Clarification & confirmation UX](#)
- g. [7\) Human handoff & audit](#)
- h. [8\) Safety, permissions, and privacy](#)
- i. [9\) Monitoring & continuous improvement](#)
- j. [10\) Implementation patterns & tech choices](#)

Q: Explain intent classification and how LLMs can be used

Intent classification (intent detection)

- Definition: mapping a user utterance to a discrete actionable label representing the speaker's goal (e.g., BookFlight, CancelOrder, AskRefund). It's a core component of task-oriented dialogue, chatbots, routing, and query understanding. [Springer](#)

Common approaches (overview)

- Traditional supervised classification: bag-of-words / TF-IDF + classical classifiers (SVM, logistic regression). Works well with lots of labeled examples. [Springer](#)
- Neural / transformer fine-tuning: add a classification head to a pretrained encoder (BERT, RoBERTa, etc.) and fine-tune on labeled intents — current standard when labeled data exists.
- Few-shot / zero-shot and prompt-based methods with LLMs: use prompts or in-context examples to have a frozen LLM classify utterances without or with very few labeled examples. Practical when labels are scarce or intents evolve.
- Retrieval/embedding + clustering: compute embeddings for utterances, cluster to discover new intents or do kNN classification for low-data settings. Useful for intent discovery and semi-supervised pipelines.
- Hybrid pipelines: combine LLM prompting, embedding retrieval, and a small supervised classifier (e.g., use LLM to generate augmented examples then fine-tune).

Q: How LLMs can be used for intent classification

- **Zero-shot / prompt classification:** craft a prompt that lists intent labels and asks the LLM to return the best label for each utterance (no training required). Fast to iterate and language-flexible; performance depends on prompt design and model capability.
- **Few-shot in-context learning:** include a few labeled example utterances in the prompt so the LLM infers the mapping; often much better than pure zero-shot for subtle intents.
- **Chain-of-thought / step-by-step prompting:** optionally ask the model to “think step by step” or show reasoning examples to improve disambiguation for hard cases (use with caution — may increase latency and verbose outputs).
- **Prompt + retrieval (RAG / semantic sampling):** retrieve semantically similar labeled examples (via embeddings) and inject them as few-shot examples into the prompt to improve robustness and domain adaptation. [arXiv](#)
- **Using LLMs to augment data:** generate paraphrases or synthetic labeled utterances for low-resource intents, then fine-tune a smaller classifier on the augmented dataset. Effective for domain jargon and expanding coverage. [Springer](#)
- **Intent discovery / clustering using LLMs:** generate candidate intent labels or descriptions from unlabeled utterances and cluster model outputs or embeddings to surface emerging intents. [arXiv](#)
- **Fine-tuning / instruction-tuning:** when privacy/latency or accuracy demands require a dedicated model, fine-tune or instruction-tune an LLM (or use parameter-efficient tuning like LoRA) for intent classification.

Practical tradeoffs and recommendations (brief)

- If you have ample labeled data: fine-tune a transformer classifier (best accuracy, predictable latency). [Springer](#)
- If labels are scarce or intents change often: use LLM zero/few-shot prompting with semantic retrieval and data augmentation.
- For production constraints (latency, cost, privacy): consider smaller fine-tuned models or run prompt-based LLMs for periodic label discovery and then train an efficient classifier.

Example minimal pipeline (practical)

1. Collect seed utterances per intent (even 5–20 each).
2. Build sentence embeddings and a kNN few-shot pool. [arXiv](#)
3. Use an LLM with prompts + retrieved few-shot examples to classify new utterances.
4. Periodically use the LLM to generate paraphrases and label candidates; add high-confidence outputs to training data.
5. Fine-tune a lightweight transformer on accumulated labeled data for production inference.

If you want, I can:

- Give example prompts for zero/few-shot classification, or

- Draft a short prompt + retrieval pipeline for your specific intents (provide a list of sample intents).

Q: What is the relation to multiturn conversations & tool/function calling?

Relation to multi-turn conversations

- **Context** matters: intent can shift across turns (user clarifications, corrections, follow-ups). Classifying each utterance in isolation misses handed-off intents or changes in dialogue state.
- **Stateful intent tracking**: maintain a dialogue state (current intent, slots, confirmations) and update it incrementally rather than overwriting on every turn; use history features (last N turns, summarized state, or embeddings of the dialogue) as model input.
- **Intent drift and hierarchical intents**: treat high-level intent (e.g., ManageBooking) separately from low-level sub-intents (ChangeDate, CancelSeat) and allow transitions within the state machine.
- **Ambiguity resolution**: use clarification prompts or assistant questions when confidence is low; leverage prior turns to disambiguate instead of immediate clarification if history already provides cues.
- **Error recovery**: support corrective turns (user says “no, I meant...”). Keep prior hypotheses and allow intent re-assignment with minimal friction.
- **Long-term context management**: summarize or compress older turns (memory) into a concise representation for models to avoid context window bloat.

Relation to tool / function calling

- **Intent → tool mapping**: treat tool invocation as one possible intent outcome (e.g., Intent: LookupFlight → call flights API). **Models should output structured intent + arguments** (slots) to trigger deterministic tool calls.
- **Structured outputs**: prefer constrained formats (JSON schema, function signature) so the LLM returns parseable arguments; validate before calling external tools.
- **Multi-step tool flows**: some intents require **multiple tool calls or chained tools** (search → disambiguate → action). **Orchestrate via a controller** that interprets model outputs and sequences calls.
- **Tool outputs feed back to dialogue**: results from tools become new context; update dialogue state and possibly reclassify intent (e.g., tool failed → ask follow-up).
- **Safety & permission checks**: treat tool calls as side effects — enforce authorization, rate limits, and input sanitization before executing.
- **Latency and partial execution**: for slow tools, provide interim responses (acknowledge action) and update when results arrive; maintain state so retries or cancellations are possible.
- **Retrieval + function calling**: use retrieval to populate function arguments (e.g., fetch user profile to fill slots) and include retrieved evidence in the prompt used to decide whether to call a tool.

How LLMs enable both in practice (concise)

- Use prompts/in-context examples that include recent turns and required function schemas so the LLM outputs both intent label and structured arguments.
- Combine retrieval (similar examples, user profile) with few-shot prompts to improve slot filling and correct tool selection.
- Use explicit system instructions or function definitions (API signature) to constrain outputs into callable formats.
- Employ a controller that: (1) validates LLM output, (2) sequences tool calls, (3) feeds tool results back into context, and (4) updates intent/state.

If you want, I can provide a concrete prompt template + JSON function schema and a short orchestration pseudocode for a multi-turn intent→tool pipeline.

Q: If a business, product or management stakeholder expects intent classification to be used at the first prompt/query from the user, how to explain and manage the expectations?

Key message (one line)

Intent classification on the very first user query is possible but uncertain — plan for probabilistic results, graceful fallbacks, and progressive refinement.

How to explain the limits (bullet points)

- Initial uncertainty: Single-turn utterances often lack context; confidence will be lower than for later turns that include clarifying info.
- Ambiguity is normal: Short or noisy queries (one word, slang, typos) map to multiple plausible intents.
- Tradeoffs: Prioritizing speed at first touch may reduce accuracy; adding verification steps increases latency and friction.
- Cost vs. performance: High-accuracy solutions (fine-tuned models, human review, richer data) cost more than simple heuristic/zero-shot pipelines.

Recommended expectation management (concrete steps)

1. Set a realistic KPI mix
 - Primary: precision at top-1 (e.g., 85%).
 - Secondary: top-3 recall or human escalation rate (target $\leq X\%$).
2. Report confidence, not just labels
 - Show automated confidence score and top N candidate intents. Use thresholds to trigger follow-ups or fallback flows.
3. Design graceful UX
 - When confidence low, ask a short clarifying question or present quick intent choices to the user.
4. Hybrid runtime policy

- Fast path: high-confidence automated handling.
- Safe path: clarification or human review for mid/low confidence or high-risk intents (payments, cancellations).

5. Progressive refinement / enrichment

- Use contextual signals available at first touch (user locale, channel, page, query params) to boost priors.
- Immediately enrich state with retrieval (user profile, recent activity) before classification if privacy/policy allows.

6. Data loop and monitoring

- Log real outcomes and misclassifications, then continuously retrain or update prompts. ***
- Track intent drift and add discovery pipelines to surface new intents.

7. Instrument A/B testing

- Experiment: pure automatic vs. confirmation UI vs. lightweight clarification to measure user satisfaction, conversion, and cost.

8. Define safety rules

- Block automated actions on high-impact intents unless confidence + verification pass.

Short example phrasing for stakeholders

"Expect an automated prediction on the first query, but treat it as probabilistic: we'll auto-handle high-confidence cases, clarify or escalate medium/low-confidence ones, and continuously improve accuracy using live data. That balances user experience, risk, and cost."

If useful, I can draft: (A) suggested KPI targets and dashboard metrics, or (B) a simple first-turn flow diagram with thresholds and confirmation copy. Which do you want?

Q: List more possible intent outcomes

Here are grouped examples of possible intent outcomes across common domains — use these as labels or to inspire hierarchical intents and slot schemas.

General / Utility

- Greet
- Goodbye
- ThankYou
- ClarifyIntent
- ProvideFeedback
- SmallTalk

- RepeatRequest
- EscalateToHuman
- UnknownIntent

Ecommerce / Retail

- BrowseCatalog
- SearchProduct
- ViewProductDetails
- AddToCart
- RemoveFromCart
- UpdateCartQuantity
- CheckoutInitiate
- ApplyPromoCode
- CalculateShipping ***
- TrackOrder
- CancelOrder
- ReturnItem
- RequestRefund
- AskProductAvailability
- CompareProducts
- RequestRecommendation

Travel / Booking

- SearchFlights
- SearchHotels
- BookFlight
- BookHotel
- ModifyBooking
- CancelBooking
- CheckIn
- SeatSelection

- RequestItinerary
- RequestVisaInfo
- PriceCheck
- RequestTravellInsurance
- LoyaltyPointsQuery

Finance / Banking

- CheckBalance
- TransferFunds
- PayBill
- SchedulePayment
- OpenAccount
- CloseAccount
- DisputeTransaction
- ReportFraud
- RequestStatement
- UpdatePersonalInfo
- LoanInquiry
- CurrencyConversion

Customer Support / SaaS

- ReportBug
- RequestFeature
- AccountRecovery
- ResetPassword
- UpdateSubscription
- UpgradePlan
- DowngradePlan
- BillingQuestion
- TechnicalTroubleshoot
- ProvideLogs

- RequestDocs
- IntegrationHelp

Communication / Productivity

- SendMessage
- ScheduleMeeting
- CancelMeeting
- RescheduleMeeting
- ShareFile
- SetReminder
- CreateTask
- UpdateTaskStatus

Healthcare / Medical (non-diagnostic)

- BookAppointment
- CancelAppointment
- RequestPrescriptionRefill
- MedicationInformation
- SymptomReport (triage)
- RequestMedicalRecords
- InsuranceQuery
- ProviderSearch

Media / Entertainment

- PlaySong
- PausePlayback
- RecommendMovie
- SearchLyrics
- CreatePlaylist
- RateContent
- ShareContent

Smart Home / IoT

- TurnOnDevice
- TurnOffDevice
- SetTemperature
- LockDoor
- UnlockDoor
- CheckDeviceStatus
- ScheduleAutomation

Legal / Compliance

- RequestContract
- SubmitComplianceDoc
- AskPolicyClarification
- ReportViolation

HR / Internal Tools

- RequestTimeOff
- SubmitExpense
- CheckPayslip
- OnboardEmployee
- UpdateHRRecord

Developer / API

- GetEndpointDocs
- DeployService
- RollbackDeployment
- FetchLogs
- CreateTicket

Discovery / Data Ops

- ExploreData
- RunQuery

- GenerateReport
- SchedulePipeline
- InvestigateAlert

Intent outcomes for orchestration / tooling (system-level)

- CallApiFunction (with schema)
- QueryDatabase
- TriggerWorkflow
- StartHumanHandoff
- DeferToAgent
- NoActionRequired

Variants and meta-labels

- AmbiguousIntent
- LowConfidence
- HighConfidence
- IntentChange
- FollowUpRequired

If you want these as a CSV, JSON schema (with example slots for each intent), or a hierarchical tree grouped for a specific product, tell me which output format and target domain.

Q: Is intent classification always followed by actioning the intent?

No.

- Intent classification is a prediction, not an automatic command. Whether it leads to an action depends on confidence, risk, UX, and policy. ***
- **Common non-action outcomes:**
 - Clarification question (low confidence or ambiguous).
 - Present choices / confirm (medium confidence or high-impact intent).
 - Surface information only (informational intents).
 - Queue for human review (high-risk or low-trust cases).
 - Log / analytics (for discovery or training).

- No-op (irrelevant, small-talk, or spam).
- Factors that decide action vs. non-action: ***
 - Confidence score and top-N candidates.
 - Action impact (financial, legal, safety).
 - Regulatory and authorization requirements.
 - UX tradeoffs (latency vs. friction).
 - Availability of required slots/parameters.
 - Business rules (cost, SLA, fraud detection).
- Typical production policy (simple):
 - If confidence high AND required slots present AND action low-risk → auto-execute.
 - If confidence medium → ask quick confirmation or present choices.
 - If confidence low or action high-risk → ask clarification or escalate to human.
- Best practice: surface confidence and provenance, validate inputs, audit actions, and maintain a safe fallback (confirmations, human handoff, or deferred execution).

Q: Compare intent classification with the classification task in machine learning

Use table for clear side-by-side comparison.

Aspect	Intent classification (NLP/dialogue)	General classification (ML)
Definition	Map a user utterance (text, sometimes multimodal/context) to a discrete conversational/actionable label (intent)	Map an input instance (vector, image, text, etc.) to one of several classes
Input nature	Short, noisy, context-dependent natural language; may include dialogue history, metadata, user profile	Structured or unstructured data; often stationary feature distributions

Context dependence	High — meaning can change with prior turns, session state, or external context	Usually independent per instance; context sometimes added but less intrinsic
Labels	Action-oriented, business/UX driven (e.g., BookFlight, RefundRequest); hierarchical and evolving	Typically task-oriented classes (spam/not spam, cat/dog); often stable and well-defined
Slot/argument needs	Often paired with slot filling (entities/parameters) and produces structured outputs for downstream actions	Usually single label output; structured outputs less common (though multilabel exists)
Ambiguity & uncertainty	High due to brevity, colloquialisms, typos; needs confidence handling and clarification flows	Varies by domain; ambiguity often from overlapping class features but less tied to interaction
Evaluation metrics	Precision/recall, top-N accuracy, confusion per intent, calibration, human escalation rate, UX metrics (task completion)	Accuracy, precision/recall, F1, ROC AUC, confusion matrix; less emphasis on UX outcomes
Operational constraints	Real-time latency, privacy, safety, authorization, integration with dialogue manager and tool calls	Varies: batch or real-time; focus on throughput, memory, and model size
Data requirements	Small labeled sets common; benefits from paraphrases, retrieval, continual labeling; intent drift requires updates	Often larger labeled datasets; class balance and feature engineering are key
Typical models	Fine-tuned transformers, few-shot LLM prompting, embedding + kNN, hybrid pipelines	Wide: logistic regression, tree ensembles, CNNs, transformers depending on data modality
Post-prediction flow ***	Can trigger actions, clarifications, tool calls, human handoff; must validate slots and permissions	Usually end of pipeline or input to downstream ML systems; fewer direct real-world side effects

Error handling	Clarify, ask follow-up, offer choices, rollback actions, escalate	Retrain, calibrate, thresholding, ensemble methods; fewer immediate UX mitigations
Discovery needs	Intent discovery and taxonomy evolution are ongoing (new intents appear)	Class set usually fixed for a task; label drift handled via retraining

Concise takeaway: intent classification is a specialized, context-rich form of classification focused on mapping language to actionable conversational outcomes and must integrate with dialogue state, slot extraction, confidence/UX policies, and tooling; general ML classification covers a broader set of data types and is often *less coupled to real-time interaction and downstream side effects*.

Q: Elaborate with concrete functional examples / use cases making the case for "context dependence" and "actions, clarifications, tool calls, human handoff; must validate slots and permissions"

Overview

Concrete examples showing why intent classification must be context-aware and why downstream steps (actions, clarifications, tool calls, human handoff, slot & permission validation) are required.

Example 1 — Ambiguous first turn that needs context

Conversation:

- User: "Change my reservation."
Possible interpretations (first turn only):
- Change date/time
- Change passenger name
- Change seat
- Change payment method

Why context matters

- If preceding turn showed the user just searched flights → likely Change date/time.
- If the user had a message about a misspelling → likely Change passenger name.

What to do

- Low confidence → ask a targeted clarification: “Which part of your reservation do you want to change: date, passenger name, seat, or payment?”
- If high confidence + required slots present → call modifyReservation API with validated fields.

Example 2 — Multi-turn disambiguation that changes intent

Conversation:

- User: “I want to move my meeting.”
- Assistant: “Which meeting?”
- User: “The one with Sarah on Friday.”

Interpretation shift

- Initial label: MoveMeeting (ambiguous which meeting).
- After context: MoveMeeting → ConfirmedMeetingID → proceed to reschedule.

Action flow

- Use retrieval (calendar) to find matches, present matched event(s), get confirmation, then call CalendarAPI to reschedule.
- Validation & permissions
- Ensure the assistant has calendar access (permission) and user is authorized to modify that event; if not, request permission or escalate.

Example 3 — Slot filling and validation before action (financial)

Conversation:

- User: “Send \$500 to Mark.”
- Intent + slots
- Intent: TransferFunds
- Slots needed: amount, recipient identifier (account/email), 2FA token
- Risks & checks
- Validate recipient (ambiguity: multiple Marks) — clarify using last name or account.
 - Check balance, fraud rules, transaction limits, and that user has granted payment permissions.
- Action policy
- If all validations pass and confidence high → call PaymentAPI.

- If high risk or missing 2FA → require confirmation and 2FA entry or escalate to human support.

Example 4 — Tool chaining with intermediate clarifications

Scenario: Booking a complex trip

- User: “Book me a trip to Tokyo next month.”

Pipeline

1. Interpret intent: BookTrip.
2. Retrieve user profile (preferences, loyalty accounts).
3. Ask clarifying questions: dates, budget, nonstop vs. stops.
4. Call FlightSearch API → present top options to user.
5. User picks one → call Booking API → call Payment API → send itinerary.

Why not single-step

- Multiple required slots; user decisions at each step; need to validate payment and traveler details; possible tool failures require fallbacks.

Example 5 — High-risk automatic actions vs. human handoff

Scenario: “Cancel my subscription and refund me.”

Considerations

- Legal/financial implications; risk of abuse.

Policy

- Medium confidence + high impact → require user confirmation and, for certain plans, human review before refund.
- System action: create a support ticket, notify human agent with conversation history and confidence/provenance metadata.

Example 6 — Context changes intent (intent drift)

Conversation:

- User: “I can’t log in.”
- Assistant: “Are you getting an error?”
- User: “Yes — it says password expired. Reset it.”

Flow

- Initial intent: TroubleshootLogin → may lead to AccountRecovery if user requests reset.
- System must support intent reclassification mid-session and trigger appropriate tool (send reset link) with authentication checks.

Practical implementation patterns

- Maintain dialogue state (current intent, slot values, confirmations, provenance).
- Use confidence thresholds: auto-execute only when confidence + validations pass.
- Present top-N candidate intents when ambiguous.
- Use retrieval to prefill slots (profile, past orders) before prompting.
- Enforce permission checks and rate limits before tool calls.
- Log and surface provenance (which prompts/examples led to the label) for auditing and human handoff.
- Provide fast clarifications rather than risky auto-actions for medium/low confidence or high impact intents.

Short takeaway

Intent classification in dialogue is inherently contextual; production systems must treat first-turn labels as probabilistic inputs, validate required slots & permissions, and route through clarifications, tool orchestration, or human handoff according to confidence and risk.

Q: Provide best ways to implement context and orchestration/flow.

Summary approach

Build a small, modular stack: Dialogue State Store + Context Encoder + Decision Controller + LLM/Model layer + Tool Orchestrator. Use clear contracts (schemas, confidences, auth) between modules.

1) Represent context reliably

- Dialogue state: store canonical fields — conversation_id, turn_index, active_intent, slot_values (value, source, timestamp, confidence), confirmations, recent_actions, provenance.
- Short-term window + long-term memory: keep last N turns verbatim (or compressed), and a summarized memory for user preferences / profile.
- Versioned summaries: periodically re-summarize long histories to fixed-size representations.

2) Encode context for models

- Context prioritization: include system prompt, user message, last assistant reply, unresolved slots, and recent tool outputs (truncate older turns).
- Compression: use semantic chunking or embedding-based summaries when context window is limited.
- Retrieval augmentation: fetch relevant docs, past similar conversations, or user profile and inject as retrieved context (with provenance).

3) Use hybrid modeling for intent + slots

- Two-tier: (A) **lightweight fast classifier** (small model/embedding + kNN) for initial intent candidates and confidence, (B) **LLM or fine-tuned model** for complex disambiguation and slot extraction. ***
- Few-shot + retrieved examples: populate LLM prompts with top-k similar labeled examples to improve accuracy.
- Schema-constrained outputs: require JSON/function output matching a schema to parse intents and slots reliably.

4) Confidence, thresholds, and policies

- Calibrated confidence scores per intent and per slot; maintain top-N candidates.
- Define thresholds: auto-execute threshold, confirm threshold, escalate threshold. Tie thresholds to intent risk levels (low/medium/high).
- Action policies: map (intent, confidence, risk, required_slots_present, permissions) → {auto_execute, ask_clarify, present_options, human_handoff}.

5) Orchestration / controller responsibilities

- **Validate model output:** schema check, slot normalization, permission check, rate limits, anti-fraud rules.
- **Planner:** translate validated intent+slots into ordered tool calls (including parallelizable calls), handle partial failures, retries, and compensating actions. ***
- Tool interface: use **explicit function signatures** / OpenAPI-like schemas so LLMs can produce direct call payloads; always run a validation layer before executing.
- Async handling: for long-running tools, return immediate ack and deliver final result when ready; keep state to allow cancel/rollback.

6) Clarification & confirmation UX

- Minimal targeted clarifications: ask only for missing or ambiguous slots; show top-N intent choices when ambiguous.
- Multi-choice quick replies: reduce friction with tappable options or suggested clarifying phrases.
- Progressive disclosure: when high impact, surface summary of planned action and require explicit confirmation.

7) Human handoff & audit

- Include conversation snapshot, model confidence, provenance, and flagged risk reasons for agents.
- Allow agents to take over, modify slots, or **rerun tool flows**. ***
- Keep immutable audit logs for actions and API calls (who/what/when).

8) Safety, permissions, and privacy

- **Pre-authorization checks before sensitive actions** (payments, cancellations, personal data).

- Mask or redact sensitive retrieved data unless necessary for the prompt; log access events.
- Enforce scope (least privilege) for tool APIs invoked by the system.

9) Monitoring & continuous improvement

- Instrument: intent accuracy, slot F1, auto-exec success rate, escalation rate, latency, user task completion.
- Closed-loop labeling: surface low-confidence or corrected cases to a review queue, re-label and retrain models periodically.
- Drift detection: monitor new utterance clusters and add discovery jobs to propose new intents.

10) Implementation patterns & tech choices

- State store: Redis or DynamoDB for low-latency state; append event store for audit.
- Embeddings + vector DB: Pinecone/Weaviate/FAISS for retrieval of examples and memory.
- Models: small transformer for fast inference; LLMs (prompted/fine-tuned) for hard disambiguation and generation. Use LoRA/PEFT for efficient tuning.
- Orchestrator: lightweight service (Node/Python) that validates, sequences tool calls, and handles retries/rollbacks.
- Observability: dashboards for KPIs and alerting on regressions.

If you want, I can next:

- Provide a JSON schema for intents + slots and example function signatures, or
- Draft controller pseudocode that enforces thresholds and sequences tool calls. Which do you prefer?